Text Classification using Multinomial vs Bernoulli Naïve Bayes (20 Newsgroups Dataset)

Aim

- To implement:
  - **Multinomial Naïve Bayes**
  - **Bernoulli Naïve Bayes**
- To compare performance using:
  - Accuracy
  - F1-score
- To understand differences in text modeling
- learn about 20 News group Dataset

Theory

Naïve Bayes

$$P(y \mid x) \propto P(y) \prod P(x_i \mid y)$$

🔹 **Key Difference**

Model	**Feature Type**
Multinomial NB	Word counts
Bernoulli NB	Word presence (0/1)

Python Program

```
# Naive Bayes: Multinomial vs Bernoulli
# 20 Newsgroups Dataset

import numpy as np
from sklearn.datasets import fetch_20newsgroups
from sklearn.model_selection import train_test_split
```

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB, BernoulliNB
from sklearn.metrics import accuracy_score, f1_score, classification_report

# ------------------------------
# 1. Load Dataset
# ------------------------------
categories = ['sci.space', 'rec.sport.baseball', 'comp.graphics']

data = fetch_20newsgroups(subset='all',
                          categories=categories,
                          remove=('headers', 'footers', 'quotes'))

X = data.data
y = data.target

# ------------------------------
# 2. Train-Test Split
# ------------------------------
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)

# ------------------------------
# 3. Feature Extraction
# ------------------------------

# Multinomial (word counts)
```

```
vectorizer_multi = CountVectorizer(stop_words='english', max_features=5000)
X_train_multi = vectorizer_multi.fit_transform(X_train)
X_test_multi = vectorizer_multi.transform(X_test)

# Bernoulli (binary features)
vectorizer_bern = CountVectorizer(stop_words='english', binary=True,
max_features=5000)
X_train_bern = vectorizer_bern.fit_transform(X_train)
X_test_bern = vectorizer_bern.transform(X_test)

# ------------------------------
# 4. Train Models
# ------------------------------

# Multinomial NB
mnb = MultinomialNB()
mnb.fit(X_train_multi, y_train)

# Bernoulli NB
bnb = BernoulliNB()
bnb.fit(X_train_bern, y_train)

# ------------------------------
# 5. Predictions
# ------------------------------
y_pred_mnb = mnb.predict(X_test_multi)
y_pred_bnb = bnb.predict(X_test_bern)
```

```
# ------------------------------
# 6. Evaluation
# ------------------------------
print("=== Multinomial NB ===")
print("Accuracy:", accuracy_score(y_test, y_pred_mnb))
print("F1 Score:", f1_score(y_test, y_pred_mnb, average='macro'))
print(classification_report(y_test, y_pred_mnb))

print("\n=== Bernoulli NB ===")
print("Accuracy:", accuracy_score(y_test, y_pred_bnb))
print("F1 Score:", f1_score(y_test, y_pred_bnb, average='macro'))
print(classification_report(y_test, y_pred_bnb))
```

Sample Output

```
=== Multinomial NB ===
Accuracy: 0.9153976311336718
F1 Score: 0.9152700646053145
              precision    recall  f1-score   support

           0       0.95      0.90      0.93       202
           1       0.93      0.92      0.93       202
           2       0.86      0.93      0.89       187

    accuracy                           0.92       591
   macro avg       0.92      0.92      0.92       591
weighted avg       0.92      0.92      0.92       591


=== Bernoulli NB ===
```

Accuracy: 0.8477157360406091

F1 Score: 0.8474953519771379

	precision	recall	f1-score	support
0	0.96	0.84	0.90	202
1	0.72	0.98	0.83	202
2	0.95	0.71	0.81	187
accuracy			0.85	591
macro avg	0.88	0.84	0.85	591
weighted avg	0.88	0.85	0.85	591